

Módulo 7 — Sistemas RAG y Gestión del Conocimiento

Arquitectura RAG, chunking, búsqueda híbrida, re-ranking, HyDE, GraphRAG y evaluación con RAGAS.

BLOQUE 1 · EL PROBLEMA QUE RAG RESUELVE

Módulo 7 — Sistemas RAG y Gestión del Conocimiento

Los LLMs tienen un problema fundamental para las aplicaciones empresariales: su conocimiento está congelado en el momento del entrenamiento, y la información que generan puede ser incorrecta (alucinaciones) o estar desactualizada. Para un asistente que responde preguntas sobre los productos de una empresa, los procedimientos internos, o los últimos desarrollos regulatorios, ninguna de estas limitaciones es aceptable. RAG (Retrieval-Augmented Generation) resuelve ambos problemas: en lugar de depender del conocimiento paramétrico del modelo, recupera los documentos relevantes para cada consulta de una base de conocimiento actualizable y los pone en el contexto del LLM para que genere la respuesta.

RAG fue introducido formalmente por Lewis et al. (Facebook AI Research, 2020) y rápidamente se convirtió en el patrón arquitectónico dominante para las aplicaciones de LLM en empresa. La razón es su elegancia: combina la capacidad de recuperación de información de los sistemas de búsqueda (que pueden mantenerse actualizados con nuevos documentos sin reentrenar el modelo) con la capacidad de generación y razonamiento de los LLMs (que pueden sintetizar información de múltiples documentos y generar respuestas en lenguaje natural). El resultado es un sistema que es factualmente preciso, actualizable, y explicable (puede citar las fuentes de cada afirmación).

La arquitectura de un sistema RAG básico

Un sistema RAG básico tiene dos fases principales. La fase de indexación (offline): los documentos de la base de conocimiento se procesan (extracción de texto de PDFs, Word, HTML, etc.), se dividen en chunks (fragmentos de texto de tamaño manejable, típicamente 200-1000 tokens), se convierten en embeddings vectoriales usando un modelo de embedding, y se almacenan en una base de datos vectorial junto con los metadatos de cada chunk (fuente, título, sección, fecha). La fase de recuperación y generación (online): cuando llega una consulta del usuario, se convierte en un embedding usando el mismo modelo de embedding, se buscan los chunks más similares (por similitud coseno) en la base de datos vectorial, los chunks recuperados se añaden al contexto del LLM junto con la consulta original, y el LLM genera la respuesta basándose en los chunks proporcionados.

Este sistema básico funciona sorprendentemente bien para muchos casos de uso, pero tiene limitaciones que los sistemas RAG avanzados resuelven: la calidad de la recuperación depende de que la consulta y los documentos relevantes tengan embeddings similares, lo que falla cuando hay desajuste semántico entre la consulta y los documentos (vocabulario técnico, acrónimos, variaciones lingüísticas), cuando los documentos relevantes están dispersos en múltiples chunks que individualmente no son suficientemente relevantes, o cuando la consulta requiere razonamiento sobre múltiples documentos de forma conjunta.

RAG avanzado: indexación, recuperación, y evaluación

Estrategias de chunking

La estrategia de chunking (cómo dividir los documentos en fragmentos para indexar) es uno de los determinantes más importantes de la calidad de un sistema RAG. El chunking por tamaño fijo (dividir cada N tokens con un overlap de M tokens entre chunks consecutivos) es simple pero no respeta la estructura del documento. El chunking por estructura semántica (dividir por párrafos, secciones, o títulos) produce chunks más cohesivos semánticamente pero de tamaño variable. El recursive chunking usa una jerarquía de divisores (primero por doble salto de línea, luego por salto de línea, luego por frase) hasta alcanzar el tamaño objetivo. El parent-child chunking mantiene chunks pequeños para la recuperación (mayor precisión) pero pasa al LLM el chunk padre más grande que los contiene (mayor contexto).

La elección del tamaño de chunk implica un trade-off fundamental: chunks pequeños (200-400 tokens) producen mejor precisión de recuperación (cada chunk es sobre un tema específico) pero menos contexto para el LLM. Chunks grandes (1000-2000 tokens) dan más contexto al LLM pero recuperan más ruido (información irrelevante junto con la relevante). El tamaño óptimo depende del tipo de documentos y del tipo de preguntas; la evaluación con RAGAS es la forma más rigurosa de determinar el tamaño óptimo para un caso de uso específico.

Búsqueda híbrida: vectorial + keyword

La búsqueda puramente vectorial falla cuando hay términos técnicos específicos, acrónimos, o nombres propios que el embedding puede no capturar bien semánticamente. La búsqueda keyword (BM25, el algoritmo de recuperación de texto estándar en los motores de búsqueda) captura estos términos exactos pero no captura la similitud semántica. La búsqueda híbrida combina ambas: recupera candidatos con búsqueda vectorial (para similitud semántica) y con búsqueda keyword (para coincidencia exacta de términos), y usa un algoritmo de re-ranking (RRF, Reciprocal Rank Fusion, o un cross-encoder) para combinar y ordenar los resultados. La búsqueda híbrida produce consistentemente mejores resultados que cualquiera de los dos enfoques por separado, especialmente para documentos técnicos y dominios especializados.

Re-ranking y HyDE

El re-ranking es la técnica de usar un modelo más preciso (pero más lento y costoso) para reordenar un conjunto inicial de candidatos recuperados por búsqueda vectorial o híbrida. Los cross-encoders (modelos que toman el par consulta+documento como input y producen un score de relevancia) son significativamente más precisos que los bi-encoders (que codifican consulta y documento por separado) pero demasiado lentos para buscar sobre millones de documentos. La solución es la pipeline retrieve-then-rerank: usar búsqueda eficiente para obtener los 20-50 candidatos más relevantes, y luego usar el cross-encoder para reordenar esos candidatos y seleccionar los 3-5 mejores para el LLM. HyDE (Hypothetical Document Embeddings) resuelve el problema del desajuste semántico generando un documento hipotético de respuesta a la consulta y usando el embedding de ese documento para recuperar los documentos reales más similares.

Evaluación con RAGAS

RAGAS (Retrieval-Augmented Generation Assessment) es el framework estándar de evaluación de sistemas RAG. Define cuatro métricas principales: Faithfulness (qué porcentaje de las afirmaciones de la respuesta están fundamentadas en los chunks recuperados), Answer Relevancy (qué tan relevante es la respuesta para la pregunta del usuario), Context Precision (qué porcentaje de los chunks recuperados son realmente relevantes para la pregunta), y Context Recall (qué porcentaje de la información relevante disponible fue recuperada). Estas métricas se calculan automáticamente usando un LLM como evaluador, lo que permite evaluar la calidad del sistema RAG a escala sin evaluación humana costosa.

RESUMEN EJECUTIVO DEL MÓDULO 7: RAG (Lewis et al., 2020) resuelve los dos problemas principales de los LLMs en empresa: conocimiento desactualizado y alucinaciones. La arquitectura básica tiene dos fases: indexación (chunking + embedding + base vectorial) y recuperación-generación (embedding de la consulta + búsqueda vectorial + LLM con contexto). Las técnicas avanzadas son: búsqueda híbrida (vectorial + BM25), re-ranking con cross-encoder, parent-child chunking, HyDE, y GraphRAG para conocimiento estructurado en grafos. La evaluación con RAGAS mide cuatro dimensiones: Faithfulness, Answer Relevancy, Context Precision, y Context Recall. Los casos de uso principales son: asistentes de documentación técnica, bases de conocimiento internas, análisis de contratos, y compliance con regulaciones. El tamaño de chunk y la estrategia de búsqueda son los hiperparámetros más importantes; optimizarlos con RAGAS es esencial.